

AI & ML Task 4

Classification Models, Evaluation Metrics & Handling Imbalanced Data

Breast Cancer Classification Project

Author: Sahil Bhatti

```
In [1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
```

```
In [2]: data = load_breast_cancer()

X = pd.DataFrame(
    data.data,
    columns=data.feature_names
)

y = data.target
```

```
In [3]: X.head()
```

```
Out[3]:
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	s
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	

5 rows × 30 columns



```
In [4]: X.shape
```

```
Out[4]: (569, 30)
```

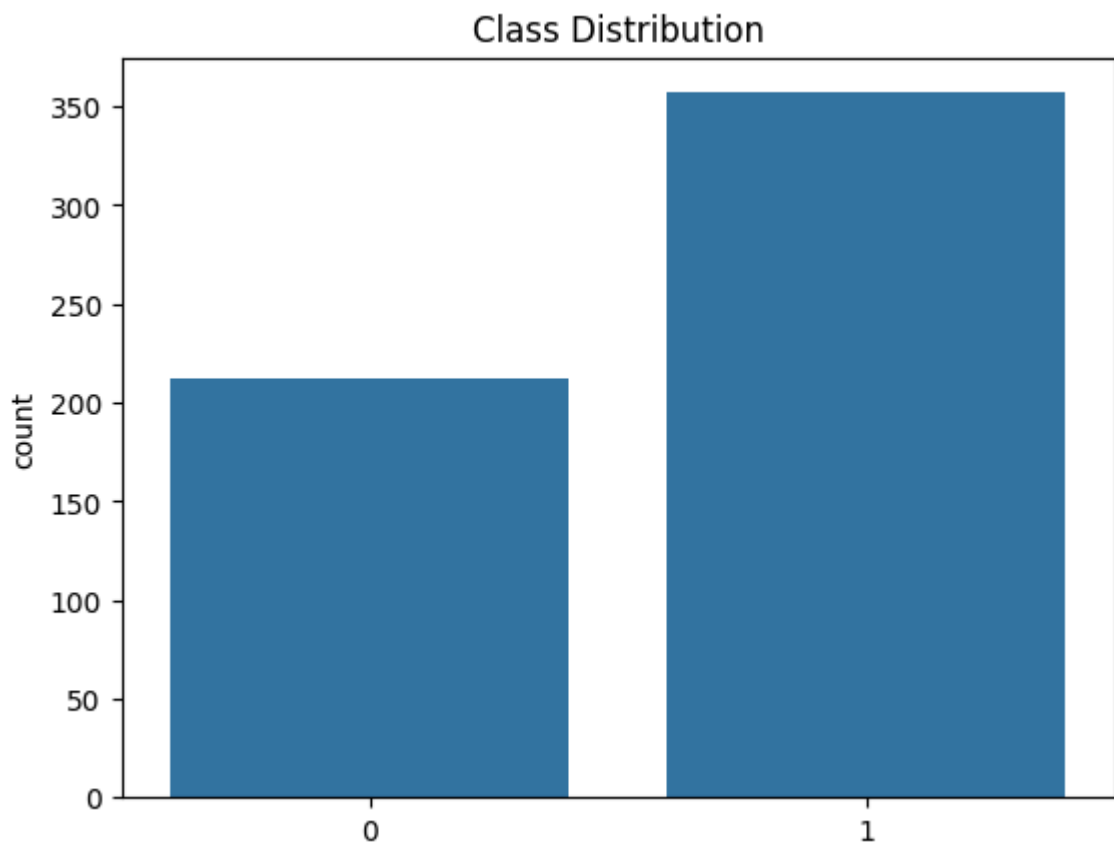
```
In [5]: y[:10]
```

```
Out[5]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
In [6]: pd.Series(y).value_counts()
```

```
Out[6]: 1    357  
        0    212  
        Name: count, dtype: int64
```

```
In [7]: sns.countplot(x=y)  
  
plt.title("Class Distribution")  
  
plt.show()
```



```
In [8]: X.isnull().sum()
```

```
Out[8]: mean radius      0
        mean texture    0
        mean perimeter   0
        mean area        0
        mean smoothness  0
        mean compactness 0
        mean concavity    0
        mean concave points 0
        mean symmetry     0
        mean fractal dimension 0
        radius error      0
        texture error     0
        perimeter error   0
        area error        0
        smoothness error  0
        compactness error 0
        concavity error   0
        concave points error 0
        symmetry error    0
        fractal dimension error 0
        worst radius      0
        worst texture     0
        worst perimeter   0
        worst area        0
        worst smoothness  0
        worst compactness 0
        worst concavity   0
        worst concave points 0
        worst symmetry     0
        worst fractal dimension 0
        dtype: int64
```

Missing Value Analysis

The dataset does not contain missing values.

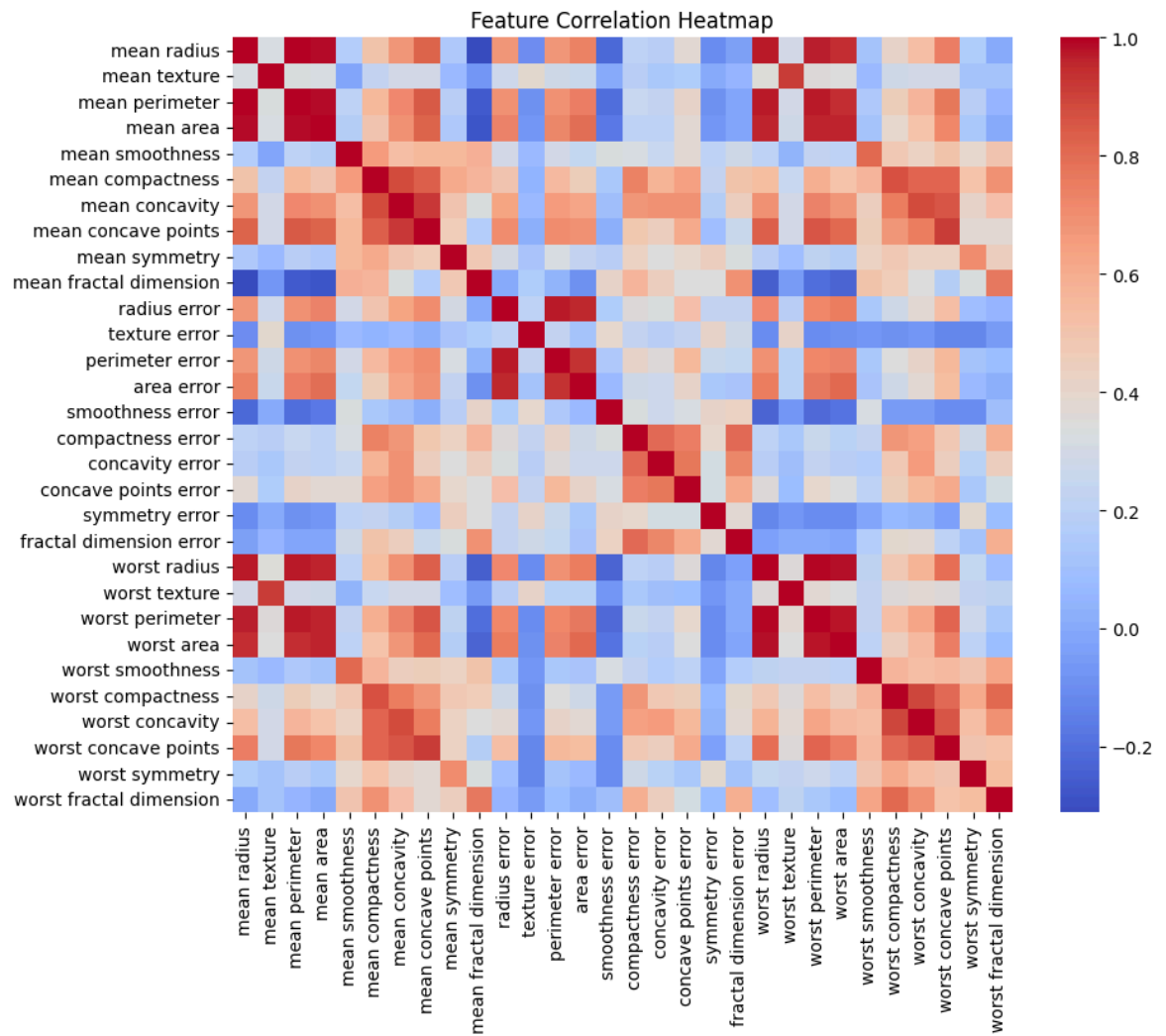
This indicates that the dataset is clean and ready for machine learning preprocessing and model building.

```
In [10]: plt.figure(figsize=(10,8))

sns.heatmap(
    X.corr(),
    cmap='coolwarm'
)

plt.title("Feature Correlation Heatmap")

plt.show()
```



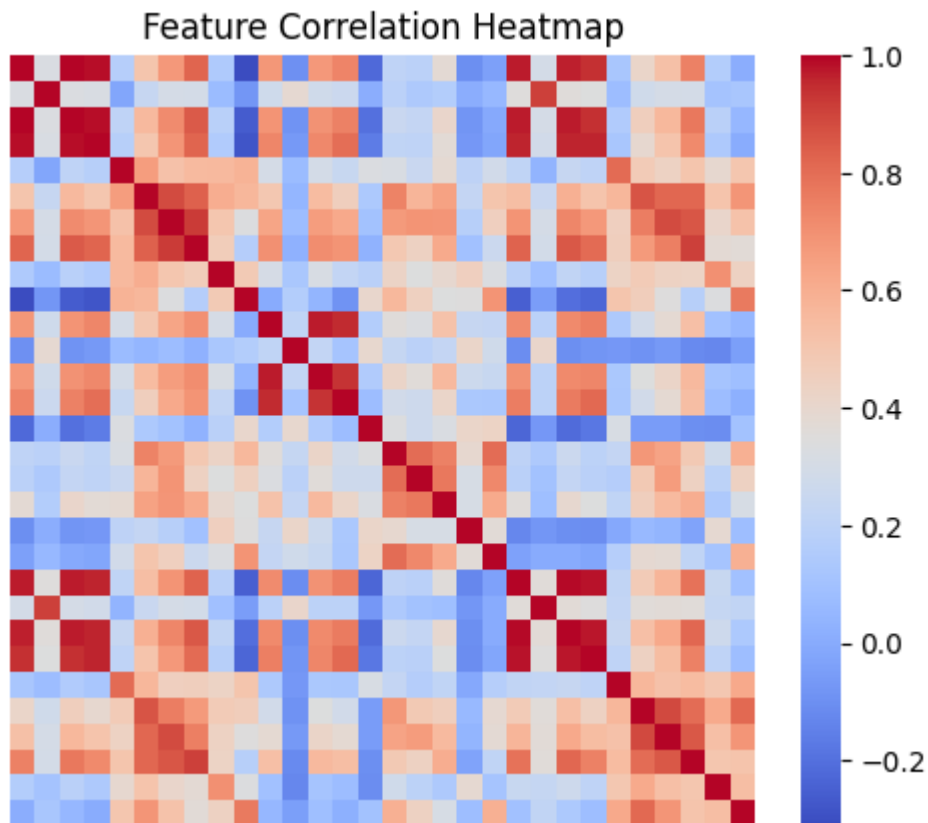
```
In [11]: plt.figure(figsize=(6,5))

sns.heatmap(
    X.corr(),
    cmap='coolwarm',
    cbar=True
)

plt.title("Feature Correlation Heatmap")

plt.xticks([])
plt.yticks([])

plt.show()
```



```
In [12]: from sklearn.model_selection import train_test_split
```

```
In [13]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
In [14]: print(X_train.shape)
print(X_test.shape)
```

```
(455, 30)
```

```
(114, 30)
```

Train-Test Split

The dataset was divided into training and testing sets.

- 80% of data was used for training
- 20% of data was used for testing

Stratification was applied to preserve class distribution.

```
In [15]: from sklearn.preprocessing import StandardScaler
```

```
In [16]: scaler = StandardScaler()
```

```
In [17]: X_train_scaled = scaler.fit_transform(X_train)
```

```
In [18]: X_test_scaled = scaler.transform(X_test)
```

Feature Scaling

Feature scaling was applied using StandardScaler.

Scaling standardizes features by transforming them to have:

- Mean = 0
- Standard Deviation = 1

This improves the performance of machine learning algorithms such as Logistic Regression.

```
In [20]: from sklearn.linear_model import LogisticRegression
```

```
In [21]: lr_model = LogisticRegression(max_iter=1000)
```

```
In [22]: lr_model.fit(  
    X_train_scaled,  
    y_train  
)
```

```
Out[22]: 

▼ LogisticRegression ⓘ ?



► Parameters


```

```
In [23]: y_pred_lr = lr_model.predict(  
    X_test_scaled  
)
```

```
In [24]: print(y_pred_lr[:10])
```

```
[0 1 0 1 0 1 1 0 0 0]
```

Logistic Regression Model

Logistic Regression was used as the baseline classification model.

The model was trained using scaled training data and then used to predict tumor classes on unseen test data.

```
In [25]: from sklearn.metrics import (  
    confusion_matrix,  
    classification_report,  
    accuracy_score  
)
```

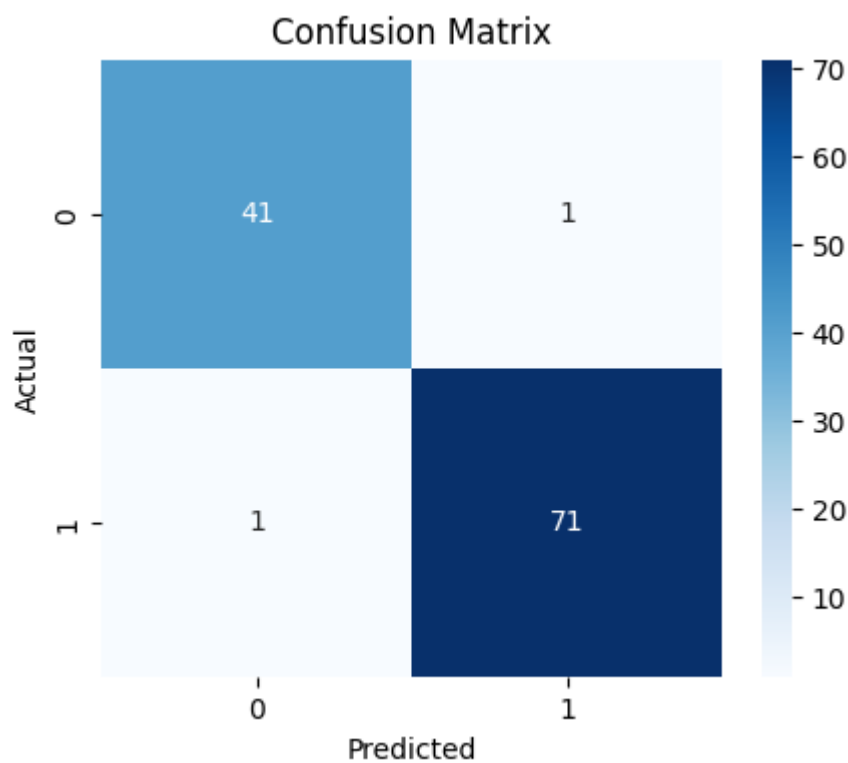
```
In [26]: accuracy_lr = accuracy_score(  
        y_test,  
        y_pred_lr  
    )  
  
    print("Accuracy:", accuracy_lr)
```

Accuracy: 0.9824561403508771

```
In [27]: cm = confusion_matrix(  
        y_test,  
        y_pred_lr  
    )  
  
    print(cm)
```

```
[[41  1]  
 [ 1 71]]
```

```
In [28]: plt.figure(figsize=(5,4))  
  
    sns.heatmap(  
        cm,  
        annot=True,  
        fmt='d',  
        cmap='Blues'  
    )  
  
    plt.title("Confusion Matrix")  
  
    plt.xlabel("Predicted")  
  
    plt.ylabel("Actual")  
  
    plt.show()
```



```
In [29]: print(
          classification_report(
              y_test,
              y_pred_lr
          )
        )
```

	precision	recall	f1-score	support
0	0.98	0.98	0.98	42
1	0.99	0.99	0.99	72
accuracy			0.98	114
macro avg	0.98	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114

Model Evaluation Metrics

Accuracy

Measures the overall correctness of predictions.

Precision

Measures how many predicted positive cases were actually positive.

Recall

Measures how many actual positive cases were correctly identified.

F1-Score

Harmonic mean of precision and recall.

Why Accuracy Alone Is Not Enough

In medical diagnosis, false negatives are dangerous because cancer cases may go undetected.

```
In [30]: from sklearn.metrics import (
          roc_curve,
          roc_auc_score
        )
```

```
In [31]: y_prob_lr = lr_model.predict_proba(
          X_test_scaled
        )[:,1]
```

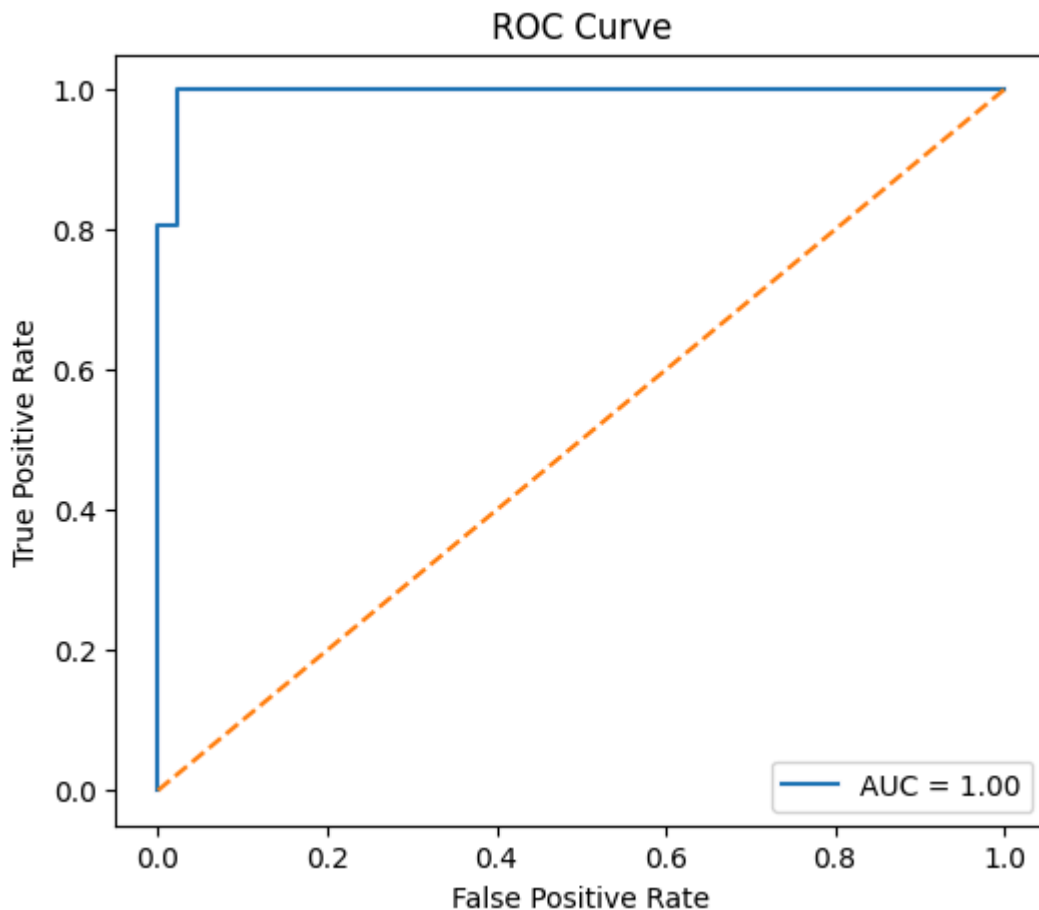


```
In [32]: fpr, tpr, _ = roc_curve(  
        y_test,  
        y_prob_lr  
    )
```

```
In [33]: auc_lr = roc_auc_score(  
        y_test,  
        y_prob_lr  
    )  
  
    print("AUC Score:", auc_lr)
```

AUC Score: 0.9953703703703703

```
In [34]: plt.figure(figsize=(6,5))  
  
    plt.plot(  
        fpr,  
        tpr,  
        label=f"AUC = {auc_lr:.2f}"  
    )  
  
    plt.plot(  
        [0,1],  
        [0,1],  
        linestyle='--'  
    )  
  
    plt.xlabel("False Positive Rate")  
  
    plt.ylabel("True Positive Rate")  
  
    plt.title("ROC Curve")  
  
    plt.legend()  
  
    plt.show()
```



ROC Curve & AUC Score

ROC Curve evaluates the classification performance of the model across different thresholds.

AUC (Area Under Curve) measures the model's ability to distinguish between classes.

AUC values closer to 1 indicate better classification performance.

```
In [35]: balanced_model = LogisticRegression(  
        class_weight='balanced',  
        max_iter=1000  
    )
```

```
In [36]: class_weight='balanced'
```

```
In [37]: balanced_model.fit(  
        X_train_scaled,  
        y_train  
    )
```

```
Out[37]: 

▼ LogisticRegression ⓘ ?



► Parameters


```

```
In [38]: y_pred_balanced = balanced_model.predict(
         X_test_scaled
        )
```

```
In [39]: print(
         classification_report(
             y_test,
             y_pred_balanced
         )
        )
```

	precision	recall	f1-score	support
0	0.91	0.98	0.94	42
1	0.99	0.94	0.96	72
accuracy			0.96	114
macro avg	0.95	0.96	0.95	114
weighted avg	0.96	0.96	0.96	114

Handling Imbalanced Data

Class imbalance occurs when one class contains significantly more samples than another.

To address this issue, class weighting was applied using:

`class_weight='balanced'`

This gives higher importance to minority class samples during training.

```
In [40]: from sklearn.tree import DecisionTreeClassifier
```

```
In [41]: tree_model = DecisionTreeClassifier(
         random_state=42
        )
```

```
In [42]: tree_model.fit(
         X_train,
         y_train
        )
```

```
Out[42]: ▼ DecisionTreeClassifier ⓘ ?
         ► Parameters
```

```
In [44]: y_pred_tree = tree_model.predict(
         X_test
        )
```

```
In [45]: accuracy_tree = accuracy_score(
         y_test,
         y_pred_tree
        )
```

```
print("Decision Tree Accuracy:", accuracy_tree)
```

Decision Tree Accuracy: 0.9122807017543859

```
In [46]: print(
    classification_report(
        y_test,
        y_pred_tree
    )
)
```

	precision	recall	f1-score	support
0	0.85	0.93	0.89	42
1	0.96	0.90	0.93	72
accuracy			0.91	114
macro avg	0.90	0.92	0.91	114
weighted avg	0.92	0.91	0.91	114

```
In [47]: cm_tree = confusion_matrix(
    y_test,
    y_pred_tree
)

plt.figure(figsize=(5,4))

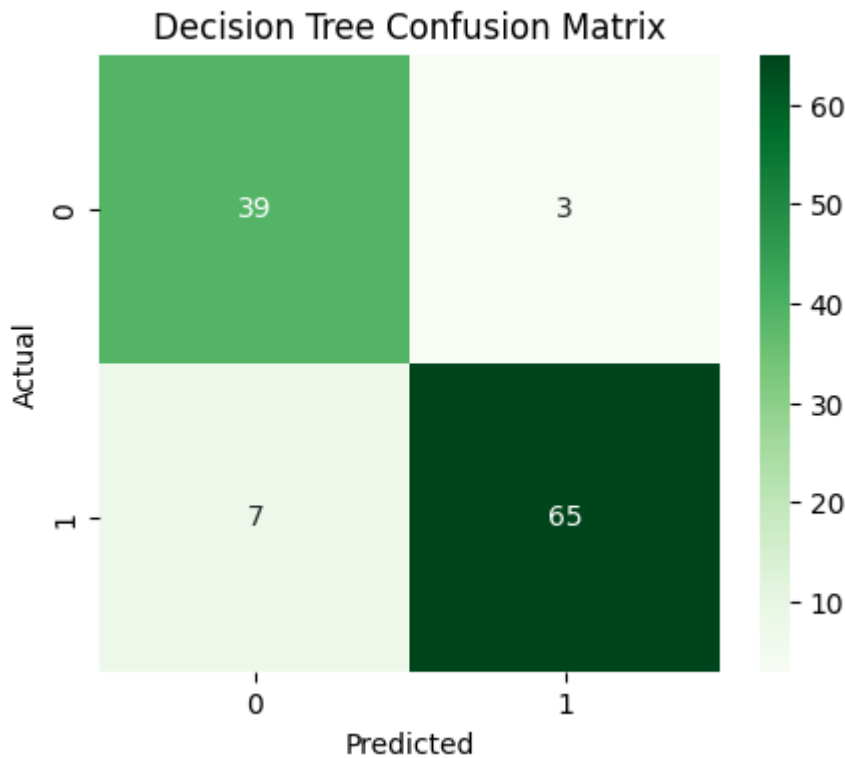
sns.heatmap(
    cm_tree,
    annot=True,
    fmt='d',
    cmap='Greens'
)

plt.title("Decision Tree Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```



Decision Tree Classifier

Decision Tree Classifier was used for model comparison.

Unlike Logistic Regression, Decision Trees create rule-based splits in the data and do not require feature scaling.

```
In [48]: from sklearn.ensemble import RandomForestClassifier
```

```
In [49]: rf_model = RandomForestClassifier(  
    random_state=42  
)
```

```
In [50]: rf_model.fit(  
    X_train,  
    y_train  
)
```

```
Out[50]: ▼ RandomForestClassifier ⓘ ?  
    ► Parameters
```

```
In [51]: y_pred_rf = rf_model.predict(  
    X_test  
)
```

```
In [52]: accuracy_rf = accuracy_score(  
    y_test,  
    y_pred_rf  
)
```

```
print("Random Forest Accuracy:", accuracy_rf)
```

Random Forest Accuracy: 0.956140350877193

```
In [53]: print(
    classification_report(
        y_test,
        y_pred_rf
    )
)
```

	precision	recall	f1-score	support
0	0.95	0.93	0.94	42
1	0.96	0.97	0.97	72
accuracy			0.96	114
macro avg	0.96	0.95	0.95	114
weighted avg	0.96	0.96	0.96	114

```
In [54]: cm_rf = confusion_matrix(
    y_test,
    y_pred_rf
)

plt.figure(figsize=(5,4))

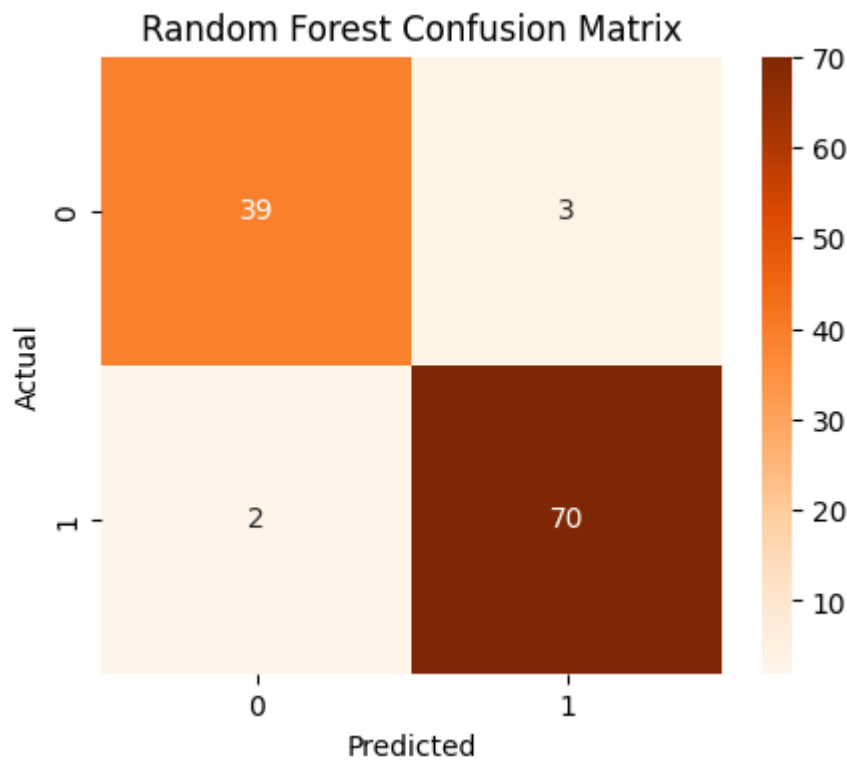
sns.heatmap(
    cm_rf,
    annot=True,
    fmt='d',
    cmap='Oranges'
)

plt.title("Random Forest Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```



Random Forest Classifier

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve prediction performance and reduce overfitting.

It generally achieves better stability and accuracy than a single Decision Tree.

```
In [55]: comparison = pd.DataFrame({  
    "Model": [  
        "Logistic Regression",  
        "Decision Tree",  
        "Random Forest"  
    ],  
    "Accuracy": [  
        accuracy_lr,  
        accuracy_tree,  
        accuracy_rf  
    ]  
})  
  
comparison
```

```
Out[55]:
```

	Model	Accuracy
0	Logistic Regression	0.982456
1	Decision Tree	0.912281
2	Random Forest	0.956140

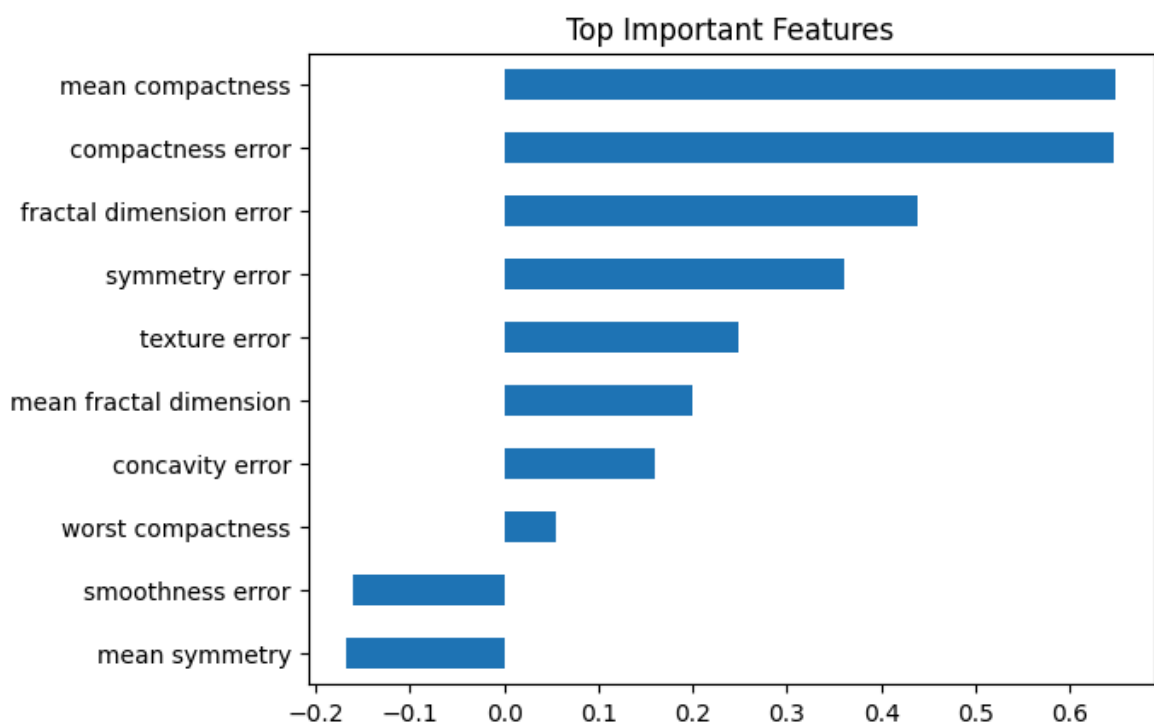
Model Comparison

Different classification models were evaluated and compared using accuracy scores.

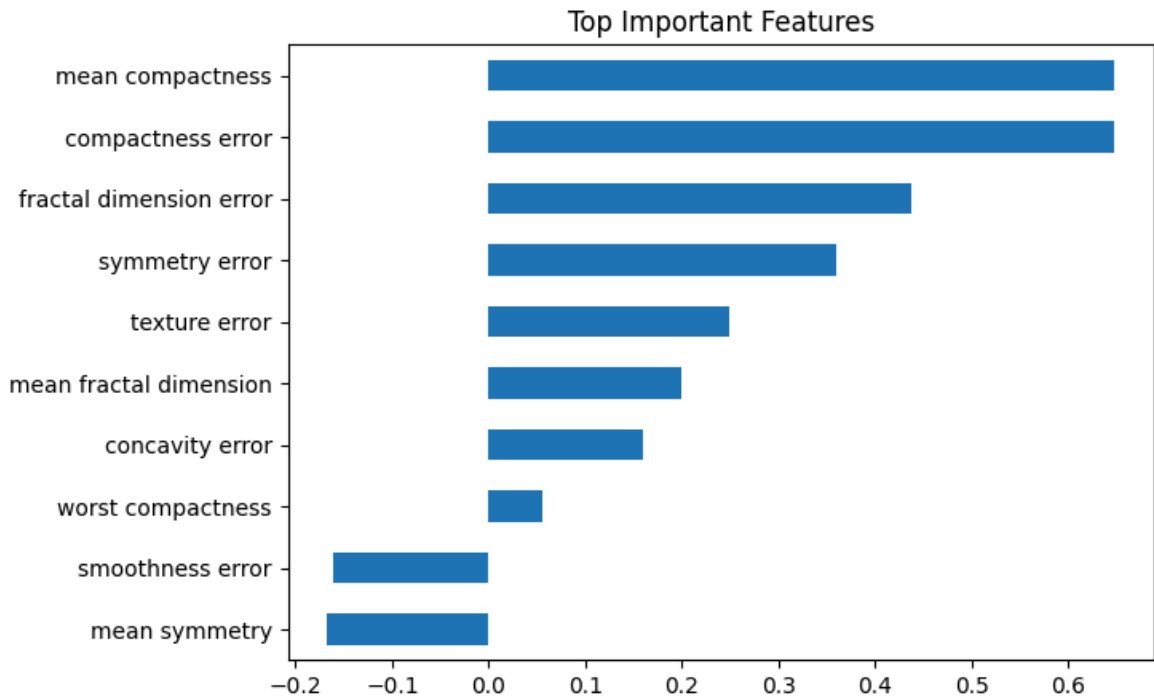
Random Forest generally achieved strong performance due to ensemble learning, while Logistic Regression provided better interpretability and stability.

```
In [56]: importance = pd.Series(  
        lr_model.coef_[0],  
        index=X.columns  
    )
```

```
In [57]: importance.sort_values().tail(10).plot(  
        kind='barh'  
    )  
  
plt.title("Top Important Features")  
  
plt.show()
```



```
In [58]: plt.figure(figsize=(7,5))  
  
importance.sort_values().tail(10).plot(  
        kind='barh'  
    )  
  
plt.title("Top Important Features")  
  
plt.show()
```

Feature Importance Analysis

Feature importance analysis was performed using Logistic Regression coefficients.

Features with larger coefficient values have stronger influence on classification predictions.

This helps identify the most significant medical indicators for tumor classification.

Final Insights

- Logistic Regression achieved strong classification performance with high accuracy and balanced precision-recall scores.
- Random Forest provided strong predictive performance due to ensemble learning and reduced overfitting.
- Recall is especially important in medical diagnosis because false negatives may lead to undetected cancer cases.
- ROC-AUC analysis showed that the models were capable of effectively distinguishing between malignant and benign tumors.
- Class imbalance handling improved the model's ability to focus on minority class samples.

Final Conclusion

In this project, multiple machine learning classification models were developed and evaluated for breast cancer prediction.

The project included:

- Data preprocessing
- Feature scaling
- Logistic Regression
- Decision Tree Classification
- Random Forest Classification
- Confusion Matrix analysis
- ROC Curve and AUC evaluation
- Imbalanced data handling

Among the evaluated models, Logistic Regression provided strong interpretability and stable classification performance, while Random Forest achieved highly effective predictive capability.

The project demonstrates the importance of evaluation metrics such as precision, recall, F1-score, and ROC-AUC in medical classification problems.

End of Project

Thank You